Kabir Kanha Arora
Rakesh Verma
CSD 330 – Security Analytics
18 April 2020

# Detection of Cyber Harassers on Social Media: Final Report

## Problem Statement

The goal is to classify a message/comment/any written text exchanged between two people as an instance of bullying or not, to detect cyber harassers and to make the internet a safer place. This idea is born out of a pressing need to curb a growing problem. Even the best of things come with their pitfalls. Growing technology and connectivity have led to an increased number of child predators and cyber stalkers online. My solution aims to efficiently nip this problem in the bud using Machine Learning approaches like Naïve Bayes, Support Vector Machines (SVM), Decision Trees, K-Nearest Neighbours, Random Forests and Logistic Regression.

## Use Cases

The use cases for this problem are virtually unlimited. Users on almost all social media websites and discussion forums can fall prey to such predators. This tool will be prototyped on a platform and then generalized to suit the requirements of all such platforms, thereby leaving a huge population much safer than they were before. Any website can use this tool to flag suspicious profiles and become a much better space for all its users.

## Motivation

Social media has opened doors for people who misuse it to undermine, harass, humiliate. Threaten and bully others. There is a lack of resources to detect and hinder its occurrence. On average, about 16% students have admitted that they have been bullied at some point in their lives, as per surveys conducted in the United States. The problem is hence indeed grave and needs to be addressed.

## Dataset

The dataset I have used for this project is the one provided at Kaggle, which was part of a contest held 8 years ago. https://www.kaggle.com/c/detecting-insults-in-social-commentary. There is a separate training data set with roughly 4000 tuples and a testing data set with about 2500 tuples. Each entry contains a date, a comment and a label, where '1' indicates that it is an insult and '0' indicates that it is neutral.

## Approach

Now, since we have just one feature that is to be used to predict the label, I start to build a vocabulary, using somewhat a **bag of words** approach. It is important to note here that this data set is off a website that has casual chats and hence, there are typos galore, The data is pretty dirty and **preprocessing** and **cleaning** takes up a major chunk of the effort.

I saw a large number of words with **front slashes and backslashes** like \xe2, which are probably escape sequences and have a special meaning, so to remove them, as they are garbage to us, I create a **regular expression** of the form:

```
slash_all = r'\s*(?:[\w_]*[/\\](?:[\w_]*[/\\])*[\w_]*)'
```

Which is then followed by:

```
comment = re.sub(slash_all,' ',comment)
```

I also **remove all special characters** except the apostrophe (') since none of them are useful to us, **trim extra whitespace** and change every character to **lower case** to avoid double counting and to ensure uniformity.

I then **tokenize** the comments, which essentially means that I break it up into a collection of words and then I remove **stop words**, which are the very common natural language words, that add little or no meaning to any sentence. After this, **stemming** is done to get all the words into their root forms to create further uniformity. I also perform **lemmatization**, which, in hindsight, was sort of useless after stemming, and just one of them should have been chosen. Collectively, these steps work to reduce the size of the vocabulary and prevent it from blowing up.

Once the data is ready to go, I run it through six classifiers – Naïve Bayes, Decision Tree, Support Vector Classifier, Random Forest, Logistic Regression and K-Nearest Neighbours. The results of all three, in order, are shown below.

**NAÏVE BAYES**

*Confusion Matrix*

```
[[1299  655]
 [ 289  404]]
```

*Classification Report*

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| 0 | 0.82 | 0.66 | 0.73 | 1954 |
| 1 | 0.38 | 0.58 | 0.46 | 693 |
| accuracy | | | 0.64 | 2647 |
| macro avg | 0.60 | 0.62 | 0.60 | 2647 |
| weighted avg | 0.70 | 0.64 | 0.66 | 2647 |

*AUC*

0.623881378487315

**DECISION TREE**

*Confusion Matrix*

```
[[1878   76]
 [ 493  200]]
```

*Classification Report*

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| 0 | 0.79 | 0.96 | 0.87 | 1954 |
| 1 | 0.72 | 0.29 | 0.41 | 693 |
| accuracy | | | 0.79 | 2647 |
| macro avg | 0.76 | 0.62 | 0.64 | 2647 |
| weighted avg | 0.77 | 0.79 | 0.75 | 2647 |

*AUC*

0.6248528566849958

**SUPPORT VECTOR CLASSIFIER (SVC)**

*Confusion Matrix*

```
[[1895   59]
 [ 442  251]]
```

*Classification Report*

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| 0 | 0.81 | 0.97 | 0.88 | 1954 |
| 1 | 0.81 | 0.36 | 0.50 | 693 |
| accuracy | | | 0.81 | 2647 |
| macro avg | 0.81 | 0.67 | 0.69 | 2647 |
| weighted avg | 0.81 | 0.81 | 0.78 | 2647 |

*AUC*

0.6659994446586054

**LOGISTIC REGRESSION**

*Confusion Matrix*

```
[[1829  125]
 [ 333  360]]
```

*Classification Report*

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| Negative | 0.85 | 0.94 | 0.89 | 1954 |
| Positive | 0.74 | 0.52 | 0.61 | 693 |
| accuracy | | | 0.83 | 2647 |
| macro avg | 0.79 | 0.73 | 0.75 | 2647 |
| weighted avg | 0.82 | 0.83 | 0.82 | 2647 |

*AUC*

0.7277545893206078

**K-NEAREST NEIGHBOURS**

*Confusion Matrix*

```
[[1776  178]
 [ 419  274]]
```

*Classification Report*

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| Negative | 0.81 | 0.91 | 0.86 | 1954 |
| Positive | 0.61 | 0.40 | 0.48 | 693 |
| accuracy | | | 0.77 | 2647 |
| macro avg | 0.71 | 0.65 | 0.67 | 2647 |
| weighted avg | 0.76 | 0.77 | 0.76 | 2647 |

*AUC*

```
0.6521436030136133
```

**RANDOM FOREST**

*Confusion Matrix*

```
[[1778  176]
 [ 320  373]]
```

*Classification Report*

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| Negative | 0.85 | 0.91 | 0.88 | 1954 |
| Positive | 0.68 | 0.54 | 0.60 | 693 |
| accuracy | | | 0.81 | 2647 |
| macro avg | 0.76 | 0.72 | 0.74 | 2647 |
| weighted avg | 0.80 | 0.81 | 0.81 | 2647 |

*AUC*

```
0.7240839451688991
```

Looking at the results, we see that in terms of **Area under Receiver Operator Characteristic Curve** and **Accuracy**, the three classifiers that stand out above the rest are – **Support Vector Classifier, Random Forest and Logistic Regression.**

So, now we shortlist these three classifiers and try to optimize them using Grid Search for a better performance. Note that since the dataset is imbalanced and we are far more interested in detecting all the cases of bullying, **recall** is the most important factor for us. We could also look at the f2-score, but since recall is readily available in the **classification report**, we shall stick to it.

For grid search, the parameters varied, and the outputs are as follows:

**SUPPORT VECTOR CLASSIFIER (SVC)**

| Parameter | Definition | Values |
|---|---|---|
| C | Regularisation Parameter. | [0.1,1, 10, 100] |
| gamma | Kernel Coefficient. | [1,0.1,0.01,0.001] |
| kernel | Kernel Type | ['rbf', 'poly', 'sigmoid'] |

*Best estimator:*

```
SVC(C=10, break_ties=False, cache_size=200, class_weight=None, coef0=0.0,
    decision_function_shape='ovr', degree=3, gamma=0.01, kernel='rbf',
    max_iter=-1, probability=False, random_state=None, shrinking=True,
    tol=0.001, verbose=False)
```

*Confusion Matrix*

```
[[1836  118]
 [ 339  354]]
```

*Classification Report*

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| 0 | 0.84 | 0.94 | 0.89 | 1954 |
| 1 | 0.75 | 0.51 | 0.61 | 693 |
| accuracy | | | 0.83 | 2647 |
| macro avg | 0.80 | 0.73 | 0.75 | 2647 |
| weighted avg | 0.82 | 0.83 | 0.82 | 2647 |

**LOGISTIC REGRESSION**

| Parameter | Definition | Values |
|---|---|---|
| C | Inverse of regularization strength | [100,10,1.0,0.1,0.01,0.001] |
| solver | Algorithm | ['newton-cg', 'lbfgs', 'liblinear','sag','saga'] |
| penalty | Norm used in penalization. | ['none', 'l1', 'l2','elasticnet'] |

*Best estimator:*

```
LogisticRegression(C=1.0, class_weight=None,dual=False,fit_intercept=True,intercept
     _scaling=1, l1_ratio=None, max_iter=100,multi_class='auto',
     n_jobs=None, penalty='l1',random_state=None, solver='liblinear',
     tol=0.0001, verbose=0,warm_start=False)
```

*Confusion Matrix*

```
[[1821  133]
 [ 332  361]]
```

*Classification Report*

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| 0 | 0.85 | 0.93 | 0.89 | 1954 |
| 1 | 0.73 | 0.52 | 0.61 | 693 |
| accuracy | | | 0.82 | 2647 |
| macro avg | 0.79 | 0.73 | 0.75 | 2647 |
| weighted avg | 0.82 | 0.82 | 0.81 | 2647 |

**RANDOM FOREST**

| Parameter | Definition | Values |
|---|---|---|
| n-estimators | Number of trees | [10, 100, 1000] |
| max_features | Number of features | ['sqrt', 'log2'] |

*Best estimator:*

```
RandomForestClassifier(bootstrap=True, ccp_alpha=0.0, class_weight=None,
      criterion='gini', max_depth=None, max_features='sqrt',
      max_leaf_nodes=None, max_samples=None,
      min_impurity_decrease=0.0, min_impurity_split=None,
      min_samples_leaf=1, min_samples_split=2,
      min_weight_fraction_leaf=0.0, n_estimators=1000,
      n_jobs=None, oob_score=False, random_state=None,
      verbose=0, warm_start=False)
```

*Confusion Matrix*

```
[[1783  171]
 [ 316  377]]
```

*Classification Report*

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| 0 | 0.85 | 0.91 | 0.88 | 1954 |
| 1 | 0.69 | 0.54 | 0.61 | 693 |
| accuracy | | | 0.82 | 2647 |
| macro avg | 0.77 | 0.73 | 0.74 | 2647 |
| weighted avg | 0.81 | 0.82 | 0.81 | 2647 |

We start to see a good increase in performance, but there's still a lot of scope for improvement. I tried to run the GridSearch in a modified manner so that instead of looking at all the metrics, it would focus on recall. I tried running the same on the Random Forest classifier, but got exactly the same result so that approach was trashed.

I also tried using XGBoost but the results were not that great without **hyperparameter tuning,** which I did again using GridSearch on XGBoost. It is worth mentioning that each gridSearch took about two hours and required a whole new level of patience. The results for that were as follows:

| Parameter | Values |
|---|---|
| eta | [0.10, 0.20, 0.30 ] |
| max_depth | [3, 4, 5] |
| min_child_weight | [1, 3, 5, 7] |
| gamma | [0.1, 0.2 , 0.3] |
| colsample_bytree | [0.3, 0.4, 0.5] |

*Best estimator:*

```
XGBClassifier(base_score=0.5, booster='gbtree', colsample_bylevel=1,
      colsample_bynode=1, colsample_bytree=0.5, eta=0.1, gamma=0.3,
      learning_rate=0.1, max_delta_step=0, max_depth=5,
      min_child_weight=1, missing=None, n_estimators=100, n_jobs=1,
      nthread=None, objective='binary:logistic', random_state=0,
      reg_alpha=0, reg_lambda=1, scale_pos_weight=1, seed=None,
      silent=None, subsample=1, verbosity=1)
```

*Confusion Matrix*

```
[[1866   88]
 [ 414  279]]
```

*Classification Report*

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| 0 | 0.82 | 0.95 | 0.88 | 1954 |
| 1 | 0.76 | 0.40 | 0.53 | 693 |
| accuracy | | | 0.81 | 2647 |
| macro avg | 0.79 | 0.68 | 0.70 | 2647 |
| weighted avg | 0.80 | 0.81 | 0.79 | 2647 |

We see that even this approach does not seem to be getting us anywhere, so we abandon it right away. Now, since the dataset is quite imbalanced (1:3), I try to look at a combination of under sampling and oversampling on the training set to improve my classifier. For this purpose, I used **SMOTETomek** on the training data and then used the best SVC classifier, as obtained earlier by GridSearch. The training data then got resampled into a balanced configuration.

Using this updated training data, I re-ran the optimal classifiers (as returned by GridSearch) and the results are as follows:

**LOGISTIC REGRESSION**

*Confusion Matrix*

```
[[1476  478]
 [ 236  457]]
```

*Classification Report*

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| 0 | 0.86 | 0.76 | 0.81 | 1954 |
| 1 | 0.49 | 0.66 | 0.56 | 693 |
| accuracy | | | 0.73 | 2647 |
| macro avg | 0.68 | 0.71 | 0.68 | 2647 |
| weighted avg | 0.76 | 0.73 | 0.74 | 2647 |

**RANDOM FOREST**

*Confusion Matrix*

```
[[1541  413]
 [ 328  365]]
```

*Classification Report*

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| 0 | 0.82 | 0.79 | 0.81 | 1954 |
| 1 | 0.47 | 0.53 | 0.50 | 693 |
| accuracy | | | 0.72 | 2647 |
| macro avg | 0.65 | 0.66 | 0.65 | 2647 |
| weighted avg | 0.73 | 0.72 | 0.73 | 2647 |

**SUPPORT VECTOR CLASSIFIER**

*Confusion Matrix*

```
[[1424  530]
 [ 238  455]]
```

*Classification Report*

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| 0 | 0.86 | 0.73 | 0.79 | 1954 |
| 1 | 0.46 | 0.66 | 0.54 | 693 |
| accuracy | | | 0.71 | 2647 |
| macro avg | 0.66 | 0.69 | 0.66 | 2647 |
| weighted avg | 0.75 | 0.71 | 0.72 | 2647 |

**Conclusions**

- ➔ It is extremely difficult to achieve a higher recall without compromising on the other metrics heavily.
- ➔ Logistic Regression and Support Vector classifiers work the best.
- ➔ Grid Search can really improve the performance of any algorithm, but it is very slow and hence Random GridSearch could be looked instead.
- ➔ It is easy to achieve high accuracy but it doesn't mean much as long as the data is imbalanced.

**Future Scope**

- This research can also be expanded to other datasets available on the internet, such as BayzickBullyingData, TextMiningAndCyberCrimeData, Formspring Data.
- The Wikipedia list of bad words could also be added to the vocabulary to enhance the classification.
- We could also include TF-IDF to get a better result.
- A semi-supervised algorithm like Label Propagation or Label Spreading may also be used.
- Majority voting could be used.

# References


## Research work

Samghabadi, Niloofar Safi, et al. "Detecting nastiness in social media." Proceedings of the First Workshop on Abusive Language Online. 2017.

Escalante, Hugo Jair, et al. "Sexual predator detection in chats with chained classifiers." Proceedings of the 4th Workshop on Computational Approaches to Subjectivity, Sentiment and Social Media Analysis. 2013.

Miah, Md Waliur Rahman, John Yearwood, and Sid Kulkarni. "Detection of child exploiting chats from a mixed chat dataset as a text classification task." Proceedings of the Australasian Language Technology Association Workshop 2011. 2011.

Huang, Qianjia, Vivek Kumar Singh, and Pradeep Kumar Atrey. "Cyber bullying detection using social and textual analysis." Proceedings of the 3rd International Workshop on Socially-Aware Multimedia. 2014.

Nandhini, B. Sri, and J. I. Sheeba. "Online social network bullying detection using intelligence techniques." Procedia Computer Science 45 (2015): 485-492.

Xu, Jun-Ming, et al. "Learning from bullying traces in social media." Proceedings of the 2012 conference of the North American chapter of the association for computational linguistics: Human language technologies. Association for Computational Linguistics, 2012.

## Websites

Agarwal, Rahul. "The 5 Sampling Algorithms Every Data Scientist Need To Know". *Towardsdatascience*, Medium, 21 July 2019, https://towardsdatascience.com/the-5-sampling-algorithms-every-data-scientist-need-to-know-43c7bc11d17c.

Brownlee, Jason. "A Gentle Introduction to XGBoost for Applied Machine Learning". *Machinelearningmastery*, 17 August 2016, https://machinelearningmastery.com/gentle-introduction-xgboost-applied-machine-learning/.

Starmer, Joshua. "XGBoost Part 2: XGBoost Trees for Classification". *Statquest*, Wordpress, 18 February 2020, https://statquest.org/2020/01/18/xgboost-part-2-xgboost-trees-for-classification/.

Dubey, Praveen. "An Introduction to Bag of Words and how to code it in Python for NLP". *Freecodecamp*, 18 December 2018, https://www.freecodecamp.org/news/an-introduction-to-bag-of-words-and-how-to-code-it-in-python-for-nlp-282e87a9da04/.

Brownlee, Jason. "Tune Hyperparameters for Classification Machine Learning Algorithms". *Machinelearningmastery*, 13 December 2019, https://machinelearningmastery.com/hyperparameters-for-classification-machine-learning-algorithms/

Badr, Will. "Having an Imbalanced Dataset? Here is how you can fix it". *Towardsdatascience*, Medium, 22 Feb 2019, https://towardsdatascience.com/having-an-imbalanced-dataset-here-is-how-you-can-solve-it-1640568947eb.

# Appendix

| | A | B | C |
|---|---|---|---|
| 1 | Insult | Date | Comment |
| 2 | 1 | 201206181 | "You fuck your dad." |
| 3 | 0 | 201205281 | "i really don't understand your point.\xa0 It seems that you are mixing apples and oranges." |
| 4 | 0 | | "A\\xc2\\xa0majority of Canadians can and has been wrong before now and will be again.\\n\\nUnless you're supportive of the idea that nothing is full proof or perfe |
| 5 | 0 | | "listen if you dont wanna get married to a man or a women DONT DO IT. what would it bother you if gay people got married stay in your lane do you let them do them. |
| 6 | 0 | 20120619( | "C\xe1c b\u1ea1n xu\u1ed1ng \u0111\u01b0\u1eddng bi\u1ec3u t\xecnh 2011 c\xf3 \xf4n ho\xe0 kh\xf4ng ? \nC\xe1c ng\u01b0 d\xe2n ng\u1ed3i cu\xed \u0111\u |
| 7 | 0 | 201206201 | "@SDL OK, but I would hope they'd sign him to a one-year contract to start with. Give him the chance to be reliable and productive, but give themselves the out if all hi |
| 8 | 0 | 20120503( | "Yeah and where are you now?" |
| 9 | 1 | | "shut the fuck up. you and the rest of your faggot friends should be burned at the stake" |
| 10 | 1 | 201205021 | "Either you are fake or extremely stupid...maybe both..." |
| 11 | 1 | 201206201 | "That you are an idiot who understands neither taxation nor women's health." |
| 12 | 0 | 20120620( | "@jdstorm dont wish him injury but it happened on its OWN and i DOUBT he's injured, he looked embarrassed to me" |
| 13 | 0 | 20120530( | "Be careful,Jimbo.OG has a fork with your name on it." |
| 14 | 0 | | "@tonnyb  Or they just don't pay attention " |
| 15 | 0 | | "Hmmm. Perhaps some who are too pig-faced to get laid and therefore have zero chance of getting pregnant from such activity hold something against women who ca |
| 16 | 0 | | "@barrettmarson Huh? Her income was $21,912. Her spending was $5,470. My math puts that at 24.9%." |
| 17 | 1 | 20120611( | "FOR SOME REASON U SOUND RETARDED. LOL. DAMN. WHERE U BEEN NEGRO" |
| 18 | 1 | 201203201 | "You with the 'racist' screen name\n\nYou are a PieceOfShit....." |
| 19 | 0 | | "Oh! & cheating as well...1966 cup was a cheat fest, & you still dare to post that comment..shame on you lol" |
| 20 | 1 | 20120320( | "your such a dickhead..." |
| 21 | 1 | 201203202 | "Your a retard go post your head up your #%&*" |
| 22 | 0 | 20120612( | "@EephusBlue\xa0Makes you want to say "Mike MacWHOgal?"" |
| 23 | 0 | 201206112 | "http://www.youtube.com/watch?v=tLYLLPHKRU4" |
| 24 | 0 | 201205022 | "And you know they've burned holes in all the carpeting." |

Fig. 1. A snippet of the training data set

*The testing data set contains an additional column called 'Usage', however it is not relevant to our project at the moment.*

| | A | B | C | D | E |
|---|---|---|---|---|---|
| 1 | Insult | Date | Comment | Usage | |
| 2 | 0 | | "THE DRUDGE REPORT\\n\\n\\n\\nYou won't see this story on foxfag forum because they suck blacks and gay 0bam | PublicTest | |
| 3 | 0 | 201206182 | "@ian21\xa0"Roger Clemens is the fucking man, and never did any fucking steroids because he is fucking awesome. | PublicTest | |
| 4 | 1 | 201206182 | "Agree with Alan you are an extremest idiot.  You are not American, we native americans are, you are an alien" | PublicTest | |
| 5 | 0 | | "Really?\\n\\nI see Marc Lamont Hill on various programs quite often, as is Al Sharpton, and Michael Moore even g | PrivateTest | |
| 6 | 0 | 20120620( | "Really suck isn't the word, when many of our nuclear power plants start melting down, it will literally be hell on ear | PrivateTest | |
| 7 | 0 | | "...after a couple guys like him rape him first...." | PrivateTest | |
| 8 | 0 | 201205292 | "Matthews mentored Perry at USC.  Bodes well..." | PrivateTest | |
| 9 | 0 | 201206191 | "How about we accept facts and patiently wait for you to back up your claim." | PrivateTest | |
| 10 | 1 | 201206091 | "To engage in an intelligent debate with you is like debating to a retarded person.  It's useless.  It looks like you're be | PublicTest | |
| 11 | 0 | 201206121 | "Same reason no one cared Obama did cocaine, which is a felony as well." | PublicTest | |
| 12 | 0 | 201206101 | "Well, Kris, you might just be surprised. Can you say Reagan Democrats? ROMNEY has a grasp of reality and the ECO | PrivateTest | |
| 13 | 0 | 201206101 | "In reality GWB put more people on food stamps because of his 100% failed economic policies!" | PrivateTest | |
| 14 | 0 | | "because the zionists will blow us up if we don't at least try!" | PrivateTest | |
| 15 | 0 | 20120528( | "Thank you.  Unfortunately on this kindle I'm limited on how far back I can go." | PrivateTest | |
| 16 | 0 | 20120610( | "CNN supports HARCORE MASS MURDERING ISLAMIST TERRORISTS!!!!\n\n\n\n\nDon\u2019t ever forget this guys! | PublicTest | |
| 17 | 0 | 20120529( | "What is your problem AW" | PrivateTest | |
| 18 | 0 | 20120529( | "Name a corporation that has told the truth." | PrivateTest | |
| 19 | 0 | 20120528( | "You know, I don't think I ever actually put my gloves in the glove box..." | PrivateTest | |
| 20 | 0 | 20120529( | "then they shouldnt get mad when they read it." | PrivateTest | |
| 21 | 0 | 20120612( | "Richards passes it to Carter. \xa0Wow. \xa0Surprising" | PrivateTest | |
| 22 | 0 | | "I\\'m a second generation German working on my Master\\'s \\xc2\\xa0so I doubt you would be accurate in saying | PublicTest | |
| 23 | 0 | | "@Fenestrate Even tough Mitt doesn't drink. I thought about Mitt Sat nite when I tried get into Slippery Noodle bar | PrivateTest | |
| 24 | 0 | 201205282 | "Then you don't understand basketball." | PrivateTest | |
| 25 | 1 | 201203201 | "GO SUCK_YOU DADDYS_COCK_OBAMA LOVING LIBERAL_FAGGOT" | PrivateTest | |
| 26 | 0 | 201206191 | "yeah!!!! fuck undying!" | PrivateTest | |
| 27 | 0 | | "Circumstances under which abortion is legal in Israel:\\nThe termination committee approves abortions, u | PrivateTest | |

Fig. 2. A snippet of the testing data set

In [6]:
```
#View dataset schema
df.head()
```

Out[6]:

| | Insult | Date | Comment |
|---|---|---|---|
| 0 | 1 | 20120618192155Z | "You fuck your dad." |
| 1 | 0 | 20120528192215Z | "i really don't understand your point.\xa0 It ... |
| 2 | 0 | NaN | "A\\xc2\\xa0majority of Canadians can and has ... |
| 3 | 0 | NaN | "listen if you dont wanna get married to a man... |
| 4 | 0 | 20120619094753Z | "C\xe1c b\u1ea1n xu\u1ed1ng \u0111\u01b0\u1edd... |

In [7]:
```
#Remove irrelevant features
df = df.drop('Date',axis=1)
```

In [56]:
```
#Check for null values
import seaborn as sb
import matplotlib
sb.heatmap(df.isnull())
```

Out[56]: <matplotlib.axes._subplots.AxesSubplot at 0x2339723cdc8>

Fig. 3. Removal of irrelevant attributes from the training data set and check for duplicates, the same is also done for the testing data set.

```
comments_test = []
for i in range(df2.shape[0]):
    comment = df2.iloc[i, 1]
    #Regular expression for words with slashes
    slash_all = r'\s*(?:[\w_]*[/\\](?:[\w_]*[/\\])*[\w_]*)'
    #Remove words with slashes
    comment = re.sub(slash_all,' ',comment)
    #Remove special characters except '
    comment = re.sub('[^\'A-Za-z]',' ',comment)
    #Remove extra whitespace
    comment = comment.strip()
    #Change to lower case
    comment = comment.lower()
    #Tokenise
    tokenised_comment = wt(comment)
    #Remove stop words
    comments_processed = []
    for word in tokenised_comment:
        if word not in set(stopwords.words('english')):
            comments_processed.append(stemmer.stem(word))
    comment_text = " ".join(comments_processed)
    comments_test.append(comment_text)
```

Fig. 4. The word manipulation code

Then, the feature matrix is created, and the training and testing data sets are transformed.

```
# Create feature matrix
from sklearn.feature_extraction.text import CountVectorizer
matrix = CountVectorizer()
```

```
X_train = matrix.fit_transform(comments).toarray()
y_train = df.iloc[:, 0]
X_test = matrix.transform(comments_test).toarray()
y_test = df2.iloc[:, 0]
```

Fig. 5. The set-up

```
print(accuracy)
```

```
0.6422364941443143
```

```
print(cm)
```

```
[[1296  658]
 [ 289  404]]
```

```
print(cr)
```

```
              precision    recall  f1-score   support

           0       0.82      0.66      0.73      1954
           1       0.38      0.58      0.46       693

    accuracy                           0.64      2647
   macro avg       0.60      0.62      0.60      2647
weighted avg       0.70      0.64      0.66      2647
```

Fig. 6. The accuracy, confusion matrix and classification report for the Naïve Bayes baseline

```
print(accuracy)
```

```
0.7842840952021156
```

```
print(cm)
```

```
[[1877   77]
 [ 494  199]]
```

```
print(cr)
```

```
              precision    recall  f1-score   support

           0       0.79      0.96      0.87      1954
           1       0.72      0.29      0.41       693

    accuracy                           0.78      2647
   macro avg       0.76      0.62      0.64      2647
weighted avg       0.77      0.78      0.75      2647
```

Fig. 7. The accuracy, confusion matrix and classification report for the Decision Tree baseline.

```
print(accuracy)
```

```
0.8111069134869664
```

```
print(cm)
```

```
[[1896   58]
 [ 442  251]]
```

```
print(cr)
```

```
              precision    recall  f1-score   support

           0       0.81      0.97      0.88      1954
           1       0.81      0.36      0.50       693

    accuracy                           0.81      2647
   macro avg       0.81      0.67      0.69      2647
weighted avg       0.81      0.81      0.78      2647
```

Fig. 8. The accuracy, confusion matrix and classification report for the Support Vector baseline.

```
print(accuracy)
```

```
0.8269739327540612
```

```
print(cm)
```

```
[[1829  125]
 [ 333  360]]
```

```
print(cr)
```

```
              precision    recall  f1-score   support

    Negative       0.85      0.94      0.89      1954
    Positive       0.74      0.52      0.61       693

    accuracy                           0.83      2647
   macro avg       0.79      0.73      0.75      2647
weighted avg       0.82      0.83      0.82      2647
```

Fig. 9. The accuracy, confusion matrix and classification report for the Logistic Regression baseline.

```
print(accuracy)
```

```
0.7744616547034379
```

```
print(cm)
```

```
[[1776  178]
 [ 419  274]]
```

```
print(cr)
```

```
              precision    recall  f1-score   support

    Negative       0.81      0.91      0.86      1954
    Positive       0.61      0.40      0.48       693

    accuracy                           0.77      2647
   macro avg       0.71      0.65      0.67      2647
weighted avg       0.76      0.77      0.76      2647
```

Fig. 10. The accuracy, confusion matrix and classification report for the KNN baseline.

```
print(accuracy)
```

```
0.8126180581790706
```

```
print(cm)
```

```
[[1778  176]
 [ 320  373]]
```

```
print(cr)
```

```
              precision    recall  f1-score   support

    Negative       0.85      0.91      0.88      1954
    Positive       0.68      0.54      0.60       693

    accuracy                           0.81      2647
   macro avg       0.76      0.72      0.74      2647
weighted avg       0.80      0.81      0.81      2647
```

Fig. 11. The accuracy, confusion matrix and classification report for the Random Forest baseline.

```
In [54]:
from sklearn.model_selection import GridSearchCV
param_grid = {'C': [0.1,1, 10, 100], 'gamma': [1,0.1,0.01,0.001],'kernel': ['rbf', 'poly', 'sigmoid']}
grid = GridSearchCV(SVC(),param_grid,refit=True,verbose=2,n_jobs=-1)
grid.fit(X_train,y_train)
print(grid.best_estimator_)
```

```
Fitting 5 folds for each of 48 candidates, totalling 240 fits

[Parallel(n_jobs=-1)]: Using backend LokyBackend with 8 concurrent workers.
[Parallel(n_jobs=-1)]: Done  25 tasks      | elapsed: 18.8min
[Parallel(n_jobs=-1)]: Done 146 tasks      | elapsed: 82.1min
[Parallel(n_jobs=-1)]: Done 240 out of 240 | elapsed: 144.6min finished

SVC(C=10, break_ties=False, cache_size=200, class_weight=None, coef0=0.0,
    decision_function_shape='ovr', degree=3, gamma=0.01, kernel='rbf',
    max_iter=-1, probability=False, random_state=None, shrinking=True,
    tol=0.001, verbose=False)
```

Fig. 12. GridSearch with SVC

```
In [22]:
from sklearn.model_selection import GridSearchCV
param_grid = {'solver' : ['newton-cg', 'lbfgs', 'liblinear','sag','saga'], 'C': [100,10,1.0,0.1,0.01,0.001],'penalty': ['none
grid = GridSearchCV(LogisticRegression(),param_grid,refit=True,verbose=10,n_jobs=-1)
grid.fit(X_train,y_train)
print(grid.best_estimator_)
```

```
Fitting 5 folds for each of 120 candidates, totalling 600 fits

[Parallel(n_jobs=-1)]: Using backend LokyBackend with 8 concurrent workers.
[Parallel(n_jobs=-1)]: Done   2 tasks      | elapsed:   37.6s
[Parallel(n_jobs=-1)]: Done   9 tasks      | elapsed:  1.0min
[Parallel(n_jobs=-1)]: Done  16 tasks      | elapsed:  2.5min
[Parallel(n_jobs=-1)]: Done  25 tasks      | elapsed:  2.6min
[Parallel(n_jobs=-1)]: Done  34 tasks      | elapsed:  2.7min
[Parallel(n_jobs=-1)]: Done  45 tasks      | elapsed:  3.5min
[Parallel(n_jobs=-1)]: Done  56 tasks      | elapsed:  4.3min
[Parallel(n_jobs=-1)]: Done  69 tasks      | elapsed:  5.2min
[Parallel(n_jobs=-1)]: Done  82 tasks      | elapsed:  5.3min
[Parallel(n_jobs=-1)]: Done  97 tasks      | elapsed:  6.1min
[Parallel(n_jobs=-1)]: Done 112 tasks      | elapsed:  6.6min
[Parallel(n_jobs=-1)]: Done 129 tasks      | elapsed:  7.6min
[Parallel(n_jobs=-1)]: Done 146 tasks      | elapsed:  8.7min
[Parallel(n_jobs=-1)]: Done 165 tasks      | elapsed: 10.2min
[Parallel(n_jobs=-1)]: Done 184 tasks      | elapsed: 10.4min
[Parallel(n_jobs=-1)]: Done 205 tasks      | elapsed: 11.8min
[Parallel(n_jobs=-1)]: Done 226 tasks      | elapsed: 13.0min
[Parallel(n_jobs=-1)]: Done 249 tasks      | elapsed: 14.3min
[Parallel(n_jobs=-1)]: Done 272 tasks      | elapsed: 15.7min
[Parallel(n_jobs=-1)]: Done 297 tasks      | elapsed: 16.5min
[Parallel(n_jobs=-1)]: Done 322 tasks      | elapsed: 18.3min
[Parallel(n_jobs=-1)]: Done 349 tasks      | elapsed: 19.3min
[Parallel(n_jobs=-1)]: Done 376 tasks      | elapsed: 20.5min
[Parallel(n_jobs=-1)]: Done 405 tasks      | elapsed: 21.8min
[Parallel(n_jobs=-1)]: Done 434 tasks      | elapsed: 23.1min
[Parallel(n_jobs=-1)]: Done 465 tasks      | elapsed: 24.2min
[Parallel(n_jobs=-1)]: Done 496 tasks      | elapsed: 24.9min
[Parallel(n_jobs=-1)]: Done 529 tasks      | elapsed: 27.4min
[Parallel(n_jobs=-1)]: Done 562 tasks      | elapsed: 27.8min
[Parallel(n_jobs=-1)]: Done 600 out of 600 | elapsed: 29.6min finished

LogisticRegression(C=1.0, class_weight=None, dual=False, fit_intercept=True,
                   intercept_scaling=1, l1_ratio=None, max_iter=100,
                   multi_class='auto', n_jobs=None, penalty='l1',
                   random_state=None, solver='liblinear', tol=0.0001, verbose=0,
                   warm_start=False)
```

Fig. 13. GridSearch with Logistic Regression

```
In [29]:
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV
param_grid = {'n_estimators' : [10, 100, 1000], 'max_features' : ['sqrt', 'log2']}
grid = GridSearchCV(RandomForestClassifier(),param_grid,refit=True,verbose=10,n_jobs=6)
grid.fit(X_train,y_train)
print(grid.best_estimator_)
```

```
Fitting 5 folds for each of 6 candidates, totalling 30 fits

[Parallel(n_jobs=6)]: Using backend LokyBackend with 6 concurrent workers.
[Parallel(n_jobs=6)]: Done   1 tasks      | elapsed:    5.8s
[Parallel(n_jobs=6)]: Done   6 tasks      | elapsed:   54.5s
[Parallel(n_jobs=6)]: Done  13 tasks      | elapsed:  1.1min
[Parallel(n_jobs=6)]: Done  23 out of  30 | elapsed:  9.5min remaining:  2.9min
[Parallel(n_jobs=6)]: Done  27 out of  30 | elapsed: 10.1min remaining:  1.1min
[Parallel(n_jobs=6)]: Done  30 out of  30 | elapsed: 11.8min finished

RandomForestClassifier(bootstrap=True, ccp_alpha=0.0, class_weight=None,
                       criterion='gini', max_depth=None, max_features='sqrt',
                       max_leaf_nodes=None, max_samples=None,
                       min_impurity_decrease=0.0, min_impurity_split=None,
                       min_samples_leaf=1, min_samples_split=2,
                       min_weight_fraction_leaf=0.0, n_estimators=1000,
                       n_jobs=None, oob_score=False, random_state=None,
                       verbose=0, warm_start=False)
```

Fig. 14. GridSearch with Random Forest

```
In [50]:  clf = xgb.XGBClassifier()
          parameters = {
               "eta"    : [0.10, 0.20, 0.30 ] ,
               "max_depth"        : [ 3, 4, 5],
               "min_child_weight" : [1, 3, 5, 7],
               "gamma"            : [0.1, 0.2 , 0.3],
               "colsample_bytree" : [0.3, 0.4, 0.5]
               }

          grid = GridSearchCV(clf,
                              parameters, n_jobs=-1, verbose=10,refit=True,scoring="neg_log_loss",cv=3)
          grid.fit(X_train, y_train)
          print(grid.best_estimator_)

          [Parallel(n_jobs=-1)]: Done 562 tasks      | elapsed: 61.1min
          [Parallel(n_jobs=-1)]: Done 597 tasks      | elapsed: 65.7min
          [Parallel(n_jobs=-1)]: Done 632 tasks      | elapsed: 70.5min
          [Parallel(n_jobs=-1)]: Done 669 tasks      | elapsed: 77.4min
          [Parallel(n_jobs=-1)]: Done 706 tasks      | elapsed: 84.2min
          [Parallel(n_jobs=-1)]: Done 745 tasks      | elapsed: 92.2min
          [Parallel(n_jobs=-1)]: Done 784 tasks      | elapsed: 96.6min
          [Parallel(n_jobs=-1)]: Done 825 tasks      | elapsed: 101.9min
          [Parallel(n_jobs=-1)]: Done 866 tasks      | elapsed: 106.7min
          [Parallel(n_jobs=-1)]: Done 909 tasks      | elapsed: 112.7min
          [Parallel(n_jobs=-1)]: Done 952 tasks      | elapsed: 118.1min
          [Parallel(n_jobs=-1)]: Done 972 out of 972 | elapsed: 120.5min finished

          XGBClassifier(base_score=0.5, booster='gbtree', colsample_bylevel=1,
                        colsample_bynode=1, colsample_bytree=0.5, eta=0.1, gamma=0.3,
                        learning_rate=0.1, max_delta_step=0, max_depth=5,
                        min_child_weight=1, missing=None, n_estimators=100, n_jobs=1,
                        nthread=None, objective='binary:logistic', random_state=0,
                        reg_alpha=0, reg_lambda=1, scale_pos_weight=1, seed=None,
                        silent=None, subsample=1, verbosity=1)
```

Fig. 15. GridSearch with XGBoost